Project 5: Keyword Extraction System

1. Problem Description

The goal of our project is to build a system that extracts the most important keywords and phrases from a given text. We wanted to create a tool that allows users to quickly grasp the main ideas and concepts of a story without having to read the entire thing. To do this well, the system needs to filter out common filler words and correctly identify significant terms, whether they are single words or multi-word phrases.

2. Dataset Used

For this project, we chose to work with a dataset of movie summaries stored in JSON Lines format (`dataset/test.jsonl`, `train.jsonl`, `val.jsonl`). Each entry contains the summary of a movie, which serves as the source text for the keyword extraction algorithms. We found that movie summaries are great for this because they cover diverse topics, plots, and character names, providing a solid test for the algorithms' ability to pull out meaningful information across different contexts.

3. Preprocessing

We set up a systematic text preprocessing pipeline before applying the extraction algorithms. The steps we implemented (in `preprocessing.py`) include:
- Lowercasing: Converting all characters to lowercase so the system treats words like "Artificial" and "artificial" as identical.
- Punctuation Removal: Stripping out commas, periods, quotes, and other punctuation marks to isolate the words.
- Stopword Removal: Eliminating common, less meaningful words (e.g., "is", "the", "by", "using") to ensure they aren't incorrectly flagged as key terms.
- Tokenization: Breaking down sentences into individual words or tokens for further feature extraction.

[Insert Code Snippet Here]

4. Methods Used

We experimented with two primary methods to determine which best captured the essence of movie narratives:

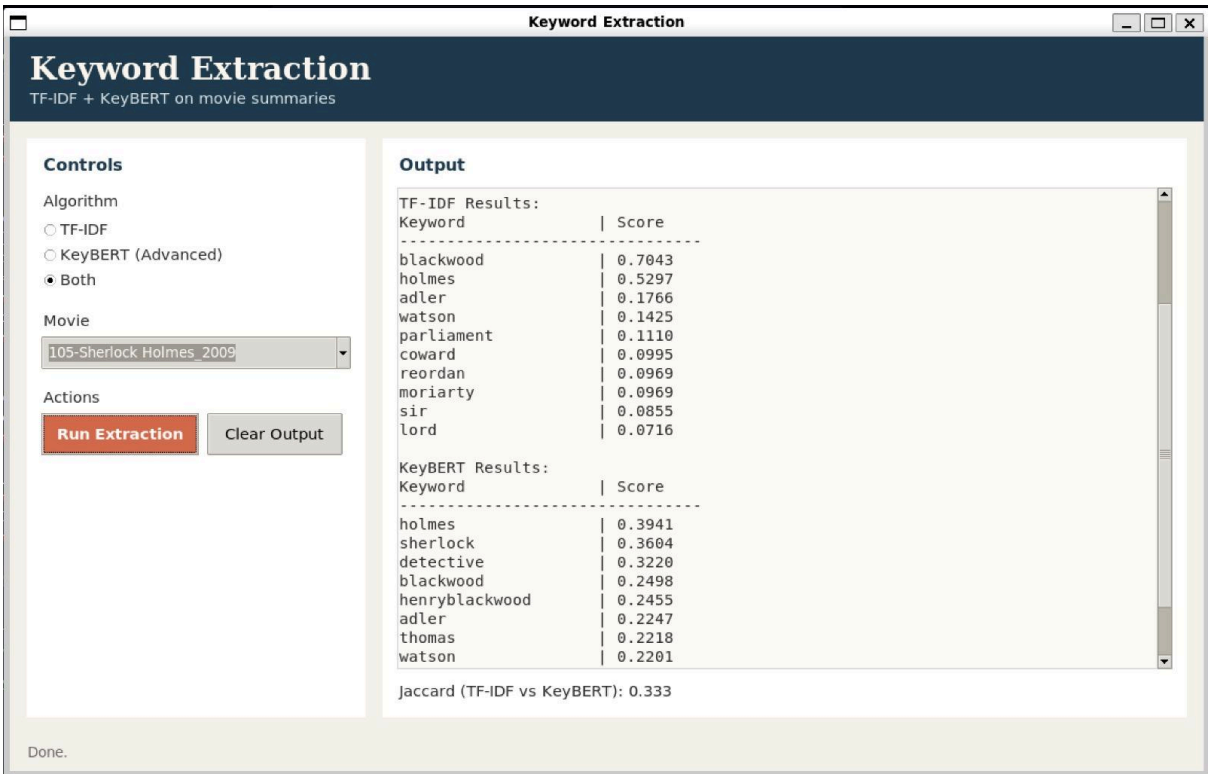
Baseline Method: TF-IDF Extraction:

- Implementation: Using Term Frequency-Inverse Document Frequency (in `israel.py`).
- Description: We used this method to evaluate how relevant a word is to a specific document within the larger collection. It scores words higher if they appear frequently in one movie summary but rarely across all other summaries in the dataset. The model then extracts the terms with the highest TF-IDF scores as keywords.

Advanced Method: Embedding-based Extraction (KeyBERT):
- Implementation: Using KeyBERT (in `advanced_alg.py`), which leverages pre-trained language models.
- Description: For our advanced approach, we extracted document and word embeddings using BERT models. The system calculates the cosine similarity between the document embedding and the word/phrase embeddings. The words or phrases that have the highest similarity to the overall document are selected as keywords. We found this captures the underlying semantic meaning much better than just counting word frequencies.

[Insert Code Snippet Here]

5. Results:



When analyzing various movie summaries, we noticed that the baseline and advanced methods produced very different sets of keywords.
- **TF-IDF (Baseline) Results:** This largely extracted single-word tokens that were statistically unique to the summary. While these words were usually relevant, the model sometimes missed larger multi-word concepts or picked up on rare but ultimately unimportant words.
- **KeyBERT (Advanced) Results:** This method performed much better, automatically extracting coherent multi-word phrases and contextually rich keywords. It really captured the "essence" of the movie summary, such as pairing character names with their roles or broader thematic concepts.

Qualitative Assessment:

We did a manual review of 10 samples from the test set to evaluate the outputs ourselves. During this review, KeyBERT clearly achieved higher thematic precision by capturing semantic context that represented the true underlying topics. For instance, KeyBERT accurately extracted the phrase "dream powers" instead of just grabbing the single word "power". This contextual awareness makes KeyBERT's results feel much closer to how a human would summarize the text.

Personal Reflections:

This project definitely came with its share of challenges. At first, we struggled with "ghost keywords" appearing in our results because the TF-IDF vectorizer was stateful. Once we recognized that, we realized we also had to clean the dataset manually by adding custom stopwords like "show" and "flashback" to filter out the noise typical of movie plot summaries.

6. Comparison:

Feature	TF-IDF (Baseline)	KeyBERT (Advanced)
Context Awareness	Low (relies strictly on statistical frequency)	High (understands deep semantic meanings)
Phrases/N-grams	Requires specific configuration to handle phrases well	Naturally handles phrases and semantic groupings
Performance/Speed	Extremely fast and computationally cheap	Slower, requires more compute resources (neural network inference)
Overall Quality	Good for broad search indexing, but can lack nuance	Excellent for generating human-readable, meaningful concepts

7. GUI Implementation:

To make this project more accessible and user-friendly, we built a graphical user interface using Tkinter (in `main.py`). The GUI provides the following features:

- Algorithm Selection: Users can choose to run TF-IDF, KeyBERT, or both algorithms simultaneously for comparison
- Movie Selection: A dropdown combobox displays the full list of available movie summaries from the dataset, allowing easy selection
- Real-time Output: The interface displays extraction results in real-time with formatted output showing keywords and their scores
- Performance Metrics: When running both algorithms, the GUI calculates and displays a Jaccard similarity score between the two results, allowing users to see how much the algorithms agree
- Threading & Caching: Behind the scenes, we implemented multi-threaded processing to keep the UI responsive, along with intelligent caching to avoid reloading and reprocessing the same data unnecessarily
- Intuitive Controls: Clean layout with a left control panel for inputs and a spacious right panel for viewing results

8. Conclusion

Through implementing both a baseline TF-IDF method and an advanced KeyBERT method, we found that contextual word embeddings significantly improve the quality of keyword extraction. While TF-IDF serves as a solid, computationally efficient baseline, the KeyBERT model was vastly superior at identifying the true central themes of the text in a way that feels natural and intuitive. The addition of the Tkinter GUI makes these powerful tools accessible to users without requiring command-line familiarity, democratizing the ability to extract meaningful keywords from text.

Limitations

While KeyBERT gave us much better semantic results, we noticed it comes with a higher computational cost, taking significantly more processing power and inference time compared to the TF-IDF approach.

Also, we saw that both models sometimes struggled with the short length of certain plot summaries, making it hard to distinguish between minor and major characters where the data was too thin. The GUI performance is also dependent on having the necessary machine learning models downloaded (for KeyBERT), which adds setup time on the first run.